EDUHUB / ENGINEERING DOCUMENTATION

LOGICAL & PHYSICAL SCHEMA

EduHub | Online Learning Platform

38	35	08
LOGICAL ENTITIES	DOMAIN TABLES	MODEL DOMAINS

Scope: relational logical schema, SQL Server physical tables, constraints and indexing.

Companion to EduHub Software Architecture and Entity-Relationship Diagram.

Proposed design based on the supplied brief and previous 38-entity ERD.

SYSTEM ANALYSIS & DESIGN / NOVEMBER 2026

Document roadmap

Implementation-level companion to the 38-entity candidate ERD, with clear boundaries between confirmed requirements and proposed SQL Server design.

38	35	08	01
MODELED ENTITIES	NON-IDENTITY TABLES	DATA DOMAINS	SQL DDL FILE

Sections

01 Modeling baseline & scope

02 Naming, types and normalization

03 Physical storage & data-integrity architecture

04 Complete logical / physical data dictionary

05 Indexing & workload strategy

06 Referential integrity and transactional rules

07 EF Core / Identity mapping and migrations

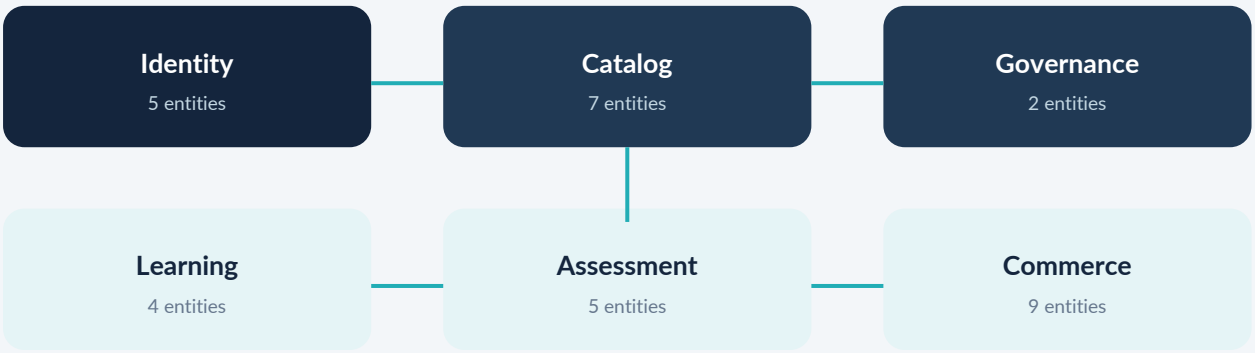
08 Source traceability and references

DOCUMENT STATUS

Candidate design for team approval. The brief prescribes EF Core, ASP.NET Identity and Azure hosting, but does not name a relational database engine. Azure SQL Database / Microsoft SQL Server is the proposed physical target. This PDF and the companion T-SQL are not evidence of an already deployed database.

01 Modeling baseline & scope

What is inherited from the project brief and prior ERD, and what the physical model decides.



Certificates (1) and Engagement (5) connect through enrollment and course ownership.

EduHub supports instructors publishing courses and lessons; students enrolling, viewing videos, tracking progress, passing quizzes and earning verifiable certificates; and administrators approving course content. Commerce includes carts, coupons, Stripe checkout, purchase history and instructor earnings. The relational model is organized around those capabilities. [S1][S2]

Requirement / design decision boundary

ITEM	STATUS	SCHEMA IMPLICATION
Three roles, courses, progress, quizzes, certificates, payment	Specified	Preserve core entity and workflow coverage.
38 logical entities in eight domains	ERD baseline	No entity added/removed; names remain traceable.
ASP.NET Core Identity	Specified	Framework-owned user/role/junction schema.
Azure SQL / SQL Server	Proposed	T-SQL types, indexes, constraints, ROWVERSION.
Identity string keys; BIGINT domain keys	Proposed	Consistent FK typing and scalable surrogates.
UTC timestamps; VARCHAR statuses	Proposed	Explicit state checks; store UTC in DATETIME2(3).

NOT IN SCOPE

The ERD is a separate deliverable. This document defines field-level logical attributes and the proposed physical implementation; it does not implement the EF Core C# entities, seed data, full API endpoints or a working deployment.

02 Naming, types and normalization

Uniform conventions prevent accidental joins, precision loss and update anomalies.

CONVENTION	PHYSICAL CHOICE	RATIONALE
Table names	dbo.PluralPascalCase	ApplicationUser -> AspNetUsers; Order -> Orders.
Application keys	BIGINT IDENTITY(1,1)	Internal surrogate key for most domain tables.
Identity keys	NVARCHAR(450)	Compatible with default string-key Identity model.
Money	DECIMAL(18,2)	Avoid binary floating-point arithmetic.
Scores	DECIMAL(5,2), DECIMAL(6,2)	Deterministic quiz scores and points.
Short text / body	NVARCHAR(n) / NVARCHAR(MAX)	Unicode UI text and long descriptions.
Identifiers / enums	VARCHAR(n)	Stripe IDs, codes and validated status strings.
Dates	DATETIME2(3) UTC	Consistent clock and precision for audits.
Flags	BIT	Non-null boolean where possible.
Concurrency	ROWVERSION	SQL Server optimistic concurrency for Course.
Public codes	VARCHAR(80) + UNIQUE	Certificate verification and order numbers.

Normalization notes

1NF: Course sections, lessons, order lines, quiz options and tags are separate row sets; no comma-separated collections in a relational column. 2NF: junction tables (UserRole, CourseTag, CartItem, LessonBookmark, ReviewHelpfulVote) use composite keys; non-key facts depend on the whole key. 3NF: descriptive fields live on their owning entity; profile data is split from the Identity principal.

Intentional snapshots: Order/OrderItem totals, titles and earning entries are retained for financial history despite the catalog changing. Derived values: average course rating and completion percentage are calculated from CourseReview and LessonProgress (or maintained as explicitly invalidated read-model caches). PreferencesJson: is proposed for rarely queried UI preferences, not for relational relations.

03 Physical storage & integrity

SQL Server / Azure SQL implementation decisions, separate from the source project requirements.

Physical storage boundaries

DATA CLASS	STORED IN SQL	STORED OUTSIDE SQL
Users / access	Identity accounts, role membership, preferences	Authentication secrets never in custom tables.
Course videos	MediaAsset.BlobKey, metadata, status	Azure Blob object bytes and CDN-delivered media.
Certificate PDF	Certificate.PdfBlobKey and verification	Generated PDF file in secured blob storage.
Payments	Provider IDs, amount, status, event dedup IDs	Card data retained by Stripe, not EduHub.
Learning state	Progress, quiz submissions, notes, bookmarks	No learner data required on CDN.

Key structural decisions

Referential actions: use ON DELETE NO ACTION for custom foreign keys. This avoids accidental deletion of payment/audit/enrollment records and SQL Server multiple-cascade-path problems. Deletions are controlled operations; published content and financial history should normally be archived or revoked.

Unique nullable links: Enrollment.OrderItemId and PaymentTransaction.StripeObjectId use filtered unique indexes (`WHERE [Column] IS NOT NULL`). Row-level consistency: foreign keys, unique indexes, and CHECK constraints guard single-table rules; multi-table ownership/currency/course-consistency rules must be validated in a single application transaction. [R1][R2][R3]

IDENTITY OWNERSHIP

AspNetUsers, AspNetRoles and AspNetUserRoles represent the ERD Identity entities. The standard ASP.NET Core Identity supporting tables (claims, logins and tokens) remain framework-managed and are deliberately outside the ERD's 38-entity count. Use one IdentityDbContext/migrations owner. [R4]

04 Logical / physical data dictionary

All 38 entities are described field-by-field using SQL Server storage types, nullability, primary and foreign keys, defaults, indexes and CHECK rules.

DOMAIN	ENTITIES	EXAMPLE TABLES
Identity	5	AspNetUsers, StudentProfiles
Catalog	7	Courses, Lessons, MediaAssets
Governance	2	CourseApprovals, CourseAnnouncements
Learning	4	Enrollments, LessonProgresses
Assessment	5	Quizzes, QuizAttempts
Certification	1	Certificates
Commerce	9	Orders, PaymentTransactions
Engagement	5	CourseReviews, Notifications

Reading key columns: PK primary key, FK foreign key, IDENTITY SQL-generated numeric identity, TOKEN optimistic-concurrency ROWVERSION. Column NULL=YES means optional. Defaults shown are T-SQL expressions. Entity names retain the prior ERD spelling; dbo. names are physical mappings.

01 Identity domain / 5 tables

ApplicationUser / dbo.AspNetUsers

IDENTITY | Shared sign-in principal for all three user roles.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	nvarchar(450)	NO	PK	—
UserName	nvarchar(256)	YES	—	—
NormalizedUserName	nvarchar(256)	YES	—	—
Email	nvarchar(256)	YES	—	—
NormalizedEmail	nvarchar(256)	YES	—	—
EmailConfirmed	bit	NO	—	0
PasswordHash	nvarchar(max)	YES	—	—
SecurityStamp	nvarchar(max)	YES	—	—
ConcurrencyStamp	nvarchar(max)	YES	—	—
DisplayName	nvarchar(160)	NO	—	—
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
IsActive	bit	NO	—	1

DESIGN RULE: Framework retains its full Identity schema; this summary shows key built-in and EduHub extension columns.

ApplicationRole / dbo.AspNetRoles

IDENTITY | Framework roles: Student, Instructor, Admin.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	nvarchar(450)	NO	PK	—
Name	nvarchar(256)	YES	—	—
NormalizedName	nvarchar(256)	YES	—	—
ConcurrencyStamp	nvarchar(max)	YES	—	—

UserRole / dbo.AspNetUserRoles

IDENTITY | ASP.NET Core Identity many-to-many membership.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
UserId	nvarchar(450)	NO	PK; FK to AspNetUsers.Id	—
RoleId	nvarchar(450)	NO	PK; FK to AspNetRoles.Id	—

StudentProfile / dbo.StudentProfiles

IDENTITY | Optional student-specific extension of an account.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
UserId	nvarchar(450)	NO	PK; FK to AspNetUsers.Id	—
Bio	nvarchar(1000)	YES	—	—
PreferencesJson	nvarchar(max)	YES	—	—
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

DESIGN RULE: UI preferences are optional JSON; use separate tables if filtering by individual preference becomes a requirement.

InstructorProfile / dbo.InstructorProfiles

IDENTITY | Optional instructor-specific profile and public presence.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
UserId	nvarchar(450)	NO	PK; FK to AspNetUsers.Id	—
Headline	nvarchar(180)	YES	—	—
Bio	nvarchar(max)	YES	—	—
PayoutAccountRef	nvarchar(180)	YES	—	—
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

DESIGN RULE: Only reference a payout provider account; do not store bank credentials or payment secrets.

02 Catalog domain / 7 tables

Category / dbo.Categories

CATALOG | Hierarchical category and subcategory taxonomy.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
ParentId	bigint	YES	FK to Categories.Id	—
Name	nvarchar(100)	NO	—	—
Slug	nvarchar(120)	NO	—	—
IsActive	bit	NO	—	1
SortOrder	int	NO	—	0

INDEXES: UNIQUE (Slug) | INDEX (ParentId, SortOrder)

CHECK: [SortOrder] >= 0

DESIGN RULE: ParentId may be NULL. Cycle prevention and maximum depth require application validation.

MediaAsset / dbo.MediaAssets

CATALOG | Private Azure Blob references and processing metadata.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
UploadedByUserId	nvarchar(450)	NO	FK to AspNetUsers.Id	—
Kind	varchar(20)	NO	—	—
BlobKey	nvarchar(500)	NO	—	—
ContentType	varchar(120)	NO	—	—
SizeBytes	bigint	NO	—	0
DurationSeconds	int	YES	—	—
ProcessingStatus	varchar(20)	NO	—	'Pending'
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: UNIQUE (BlobKey) | INDEX (UploadedByUserId, CreatedAt)

CHECK: [Kind] IN ('Video','Image','Certificate') ; [ProcessingStatus] IN ('Pending','Processing','Ready','Failed') ; [SizeBytes] >= 0 ; [DurationSeconds] IS NULL OR [DurationSeconds] >= 0

DESIGN RULE: BlobKey is stable; signed playback URLs are issued at request time and never persisted.

Tag / dbo.Tags

CATALOG | Admin-managed reusable catalog tag.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
Name	nvarchar(80)	NO	—	—
Slug	nvarchar(100)	NO	—	—

INDEXES: UNIQUE (Slug)

Course / dbo.Courses

CATALOG | Instructor-owned course, published after approval.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
InstructorUserId	nvarchar(450)	NO	FK to InstructorProfiles.UserId	—
CategoryId	bigint	NO	FK to Categories.Id	—
ThumbnailMediaId	bigint	YES	FK to MediaAssets.Id	—
Title	nvarchar(200)	NO	—	—
Slug	nvarchar(220)	NO	—	—
Description	nvarchar(max)	NO	—	—
Level	varchar(20)	NO	—	—
PriceAmount	decimal(18,2)	NO	—	0
Currency	char(3)	NO	—	—
Status	varchar(24)	NO	—	'Draft'
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
PublishedAt	datetime2(3)	YES	—	—
RowVersion	rowversion	NO	TOKEN	—

INDEXES: UNIQUE (Slug) | INDEX (InstructorUserId, Status) | INDEX (CategoryId, Status)

CHECK: [Level] IN ('Beginner','Intermediate','Advanced','AllLevels') ; [Status] IN ('Draft','PendingApproval','Published','Rejected','Archived') ; [PriceAmount] >= 0

DESIGN RULE: PriceAmount is the current catalog price. Purchased price is frozen separately in OrderItem.

CourseSection / dbo.CourseSections

CATALOG | Course module with stable position within a course.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
CourseId	bigint	NO	FK to Courses.Id	—
Title	nvarchar(180)	NO	—	—
Description	nvarchar(1000)	YES	—	—
SortOrder	int	NO	—	—

INDEXES: UNIQUE (CourseId, SortOrder)

CHECK: [SortOrder] >= 0

Lesson / dbo.Lessons

CATALOG | Ordered lesson and optional uploaded video asset.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
SectionId	bigint	NO	FK to CourseSections.Id	—
VideoMediaId	bigint	YES	FK to MediaAssets.Id	—
Title	nvarchar(180)	NO	—	—
ContentHtml	nvarchar(max)	YES	—	—
DurationSeconds	int	NO	—	0
SortOrder	int	NO	—	—
IsPreview	bit	NO	—	0

INDEXES: UNIQUE (SectionId, SortOrder) | INDEX (VideoMediaId)

CHECK: [SortOrder] >= 0 ; [DurationSeconds] >= 0

DESIGN RULE: Validate sanitized ContentHtml and restrict uploaded asset type in the services layer.

CourseTag / dbo.CourseTags

CATALOG | Many-to-many mapping of courses to tags.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
CourseId	bigint	NO	PK; FK to Courses.Id	—
TagId	bigint	NO	PK; FK to Tags.Id	—

INDEXES: INDEX (TagId, CourseId)

03 Governance domain / 2 tables

CourseApproval / dbo.CourseApprovals

GOVERNANCE | Auditable instructor submission and admin decision.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
CourseId	bigint	NO	FK to Courses.Id	—
ReviewerUserId	nvarchar(450)	YES	FK to AspNetUsers.Id	—
Decision	varchar(20)	NO	—	'Pending'
Comments	nvarchar(1000)	YES	—	—
RequestedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
DecidedAt	datetime2(3)	YES	—	—

INDEXES: INDEX (CourseId, RequestedAt)

CHECK: [Decision] IN ('Pending','Approved','Rejected')

DESIGN RULE: Admin membership and publish transition are enforced as authorization/business rules.

CourseAnnouncement / dbo.CourseAnnouncements

GOVERNANCE | Instructor announcements for course learners.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
CourseId	bigint	NO	FK to Courses.Id	—
AuthorUserId	nvarchar(450)	NO	FK to AspNetUsers.Id	—
Title	nvarchar(180)	NO	—	—
Body	nvarchar(max)	NO	—	—
PublishedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: INDEX (CourseId, PublishedAt)

04 Learning domain / 4 tables

Enrollment / dbo.Enrollments

LEARNING | Unique student-course entitlement, optionally linked to paid line.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
StudentUserId	nvarchar(450)	NO	FK to StudentProfiles.UserId	—
CourseId	bigint	NO	FK to Courses.Id	—
OrderItemId	bigint	YES	FK to OrderItems.Id	—
Status	varchar(20)	NO	—	'Active'
EnrolledAt	datetime2(3)	NO	—	SYSUTCDATETIME()
CompletedAt	datetime2(3)	YES	—	—

INDEXES: UNIQUE (StudentUserId, CourseId) | UNIQUE (OrderItemId) FILTERED | INDEX (CourseId, Status)

CHECK: [Status] IN ('Active','Completed','Revoked')

DESIGN RULE: For free courses OrderItemId is NULL. Paid enrollment is granted only after verified fulfillment; purchase/course consistency is validated transactionally.

LessonProgress / dbo.LessonProgresses

LEARNING | One progress record for each enrolled lesson.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
EnrollmentId	bigint	NO	FK to Enrollments.Id	—
LessonId	bigint	NO	FK to Lessons.Id	—
LastPositionSeconds	int	NO	—	0
IsCompleted	bit	NO	—	0
CompletedAt	datetime2(3)	YES	—	—
UpdatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: UNIQUE (EnrollmentId, LessonId) | INDEX (LessonId)

CHECK: [LastPositionSeconds] >= 0

DESIGN RULE: The enrollment course must equal the lesson section course; cannot be expressed with these simple FKs alone.

LessonNote / dbo.LessonNotes

LEARNING | Student notes positioned within a lesson video.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
EnrollmentId	bigint	NO	FK to Enrollments.Id	—
LessonId	bigint	NO	FK to Lessons.Id	—
TimeSeconds	int	NO	—	0
Body	nvarchar(2000)	NO	—	—
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
UpdatedAt	datetime2(3)	YES	—	—

INDEXES: INDEX (EnrollmentId, LessonId, TimeSeconds)

CHECK: [TimeSeconds] >= 0

LessonBookmark / dbo.LessonBookmarks

LEARNING | One saved lesson per student enrollment.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
EnrollmentId	bigint	NO	PK; FK to Enrollments.Id	—
LessonId	bigint	NO	PK; FK to Lessons.Id	—
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: INDEX (LessonId)

05 Assessment domain / 5 tables

Quiz / dbo.Quizzes

ASSESSMENT | A published quiz for a whole course or one section.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
CourseId	bigint	NO	FK to Courses.Id	—
SectionId	bigint	YES	FK to CourseSections.Id	—
Title	nvarchar(160)	NO	—	—
PassScore	decimal(5,2)	NO	—	60
MaxAttempts	smallint	YES	—	—
IsPublished	bit	NO	—	0

INDEXES: INDEX (CourseId, SectionId)

CHECK: [PassScore] >= 0 AND [PassScore] <= 100 ; [MaxAttempts] IS NULL OR [MaxAttempts] > 0

DESIGN RULE: SectionId is optional; validate that SectionId.CourseId == Quiz.CourseId.

QuizQuestion / dbo.QuizQuestions

ASSESSMENT | Assessment prompt, type and grading weight.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
QuizId	bigint	NO	FK to Quizzes.Id	—
QuestionType	varchar(20)	NO	—	—
Prompt	nvarchar(max)	NO	—	—
CorrectText	nvarchar(500)	YES	—	—
Points	decimal(6,2)	NO	—	1
SortOrder	int	NO	—	—

INDEXES: UNIQUE (QuizId, SortOrder)

CHECK: [QuestionType] IN ('MultipleChoice','TrueFalse','FillBlank') ; [Points] > 0 ; [SortOrder] >= 0

DESIGN RULE: CorrectText is grading-only and must never be returned in student-facing quiz payloads.

QuizOption / dbo.QuizOptions

ASSESSMENT | Selectable answer for MCQ or true/false.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
QuestionId	bigint	NO	FK to QuizQuestions.Id	—
Text	nvarchar(500)	NO	—	—
IsCorrect	bit	NO	—	0
SortOrder	int	NO	—	—

INDEXES: UNIQUE (QuestionId, SortOrder)

CHECK: [SortOrder] >= 0

DESIGN RULE: Exactly one/multiple correct options is enforced by question-type validation at publish time.

QuizAttempt / dbo.QuizAttempts

ASSESSMENT | Submitted attempt with per-attempt outcome.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
QuizId	bigint	NO	FK to Quizzes.Id	—
EnrollmentId	bigint	NO	FK to Enrollments.Id	—
AttemptNumber	smallint	NO	—	—
ScorePercentage	decimal(5,2)	YES	—	—
Status	varchar(20)	NO	—	'InProgress'
StartedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
SubmittedAt	datetime2(3)	YES	—	—

INDEXES: UNIQUE (QuizId, EnrollmentId, AttemptNumber) | INDEX (EnrollmentId, Status)

CHECK: [AttemptNumber] > 0 ; [ScorePercentage] IS NULL OR ([ScorePercentage] >= 0 AND [ScorePercentage] <= 100) ; [Status] IN ('InProgress','Submitted','Graded')

DESIGN RULE: Quiz.CourseId must equal Enrollment.CourseId. Submission grade and status are calculated in one transaction.

QuizAnswer / dbo.QuizAnswers

ASSESSMENT | One recorded answer per question in an attempt.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
AttemptId	bigint	NO	FK to QuizAttempts.Id	—
QuestionId	bigint	NO	FK to QuizQuestions.Id	—
OptionId	bigint	YES	FK to QuizOptions.Id	—
TextAnswer	nvarchar(500)	YES	—	—
PointsEarned	decimal(6,2)	NO	—	0

INDEXES: UNIQUE (AttemptId, QuestionId) | INDEX (QuestionId)

CHECK: [PointsEarned] >= 0

DESIGN RULE: Ensure selected OptionId belongs to QuestionId, and QuestionId belongs to Attempt.QuizId.

06 Certification domain / 1 tables

Certificate / dbo.Certificates

CERTIFICATION | One issued credential per completed enrollment.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
EnrollmentId	bigint	NO	FK to Enrollments.Id	—
VerifyCode	varchar(80)	NO	—	—
PdfBlobKey	nvarchar(500)	NO	—	—
IssuedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
IsRevoked	bit	NO	—	0
RevokedAt	datetime2(3)	YES	—	—

INDEXES: UNIQUE (EnrollmentId) | UNIQUE (VerifyCode)

DESIGN RULE: Issue after completion and passing rules; verify only minimal public details.

07 Commerce domain / 9 tables

Cart / dbo.Carts

COMMERCE | One current shopping cart per buyer.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
UserId	nvarchar(450)	NO	FK to AspNetUsers.Id	—
UpdatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: UNIQUE (UserId)

CartItem / dbo.CartItems

COMMERCE | Course selections before checkout.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
CartId	bigint	NO	PK; FK to Carts.Id	—
CourseId	bigint	NO	PK; FK to Courses.Id	—
AddedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: INDEX (CourseId)

Order / dbo.Orders

COMMERCE | Immutable checkout totals after order placement.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
BuyerUserId	nvarchar(450)	NO	FK to AspNetUsers.Id	—
OrderNumber	varchar(40)	NO	—	—
Currency	char(3)	NO	—	—
SubtotalAmount	decimal(18,2)	NO	—	0
DiscountAmount	decimal(18,2)	NO	—	0
TotalAmount	decimal(18,2)	NO	—	0
Status	varchar(20)	NO	—	'Pending'
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
PaidAt	datetime2(3)	YES	—	—

INDEXES: UNIQUE (OrderNumber) | INDEX (BuyerUserId, CreatedAt)

CHECK: [SubtotalAmount] >= 0 ; [DiscountAmount] >= 0 ; [TotalAmount] >= 0 ; [TotalAmount] = [SubtotalAmount] - [DiscountAmount] ; [Status] IN ('Pending','Paid','Failed','Canceled','Refunded')

DESIGN RULE: Order totals are confirmed in a single transaction. Avoid editing purchase snapshots after payment.

OrderItem / dbo.OrderItems

COMMERCE | Course and final price snapshot captured at checkout.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
OrderId	bigint	NO	FK to Orders.Id	—
CourseId	bigint	NO	FK to Courses.Id	—
InstructorUserId	nvarchar(450)	NO	FK to InstructorProfiles.UserId	—
TitleSnapshot	nvarchar(200)	NO	—	—
UnitPriceAmount	decimal(18,2)	NO	—	—
DiscountAmount	decimal(18,2)	NO	—	0
FinalAmount	decimal(18,2)	NO	—	—

INDEXES: UNIQUE (OrderId, CourseId) | INDEX (CourseId)

CHECK: [UnitPriceAmount] >= 0 ; [DiscountAmount] >= 0 ; [FinalAmount] >= 0 ; [FinalAmount] = [UnitPriceAmount] - [DiscountAmount]

DESIGN RULE: Line amounts are historical snapshots, not references back to mutable Course.PriceAmount.

PaymentTransaction / dbo.PaymentTransactions

COMMERCE | Provider payment attempts, retries and outcomes.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
OrderId	bigint	NO	FK to Orders.Id	—
Provider	varchar(20)	NO	—	'Stripe'
StripeObjectId	varchar(255)	YES	—	—
Amount	decimal(18,2)	NO	—	—
Currency	char(3)	NO	—	—
Status	varchar(20)	NO	—	'Pending'
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
CompletedAt	datetime2(3)	YES	—	—

INDEXES: UNIQUE (StripeObjectId) FILTERED | INDEX (OrderId, CreatedAt)

CHECK: [Amount] >= 0 ; [Provider] IN ('Stripe') ; [Status] IN ('Pending','Succeeded','Failed','Canceled','Refunded')

DESIGN RULE: One order may have multiple attempts. StripeObjectId is UNIQUE when present.

PaymentWebhookEvent / dbo.PaymentWebhookEvents

COMMERCE | Durable Stripe delivery deduplication and processing audit.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
PaymentTransactionId	bigint	YES	FK to PaymentTransactions.Id	—
StripeEventId	varchar(255)	NO	—	—
EventType	varchar(120)	NO	—	—
PayloadHash	char(64)	YES	—	—
Status	varchar(20)	NO	—	'Received'
ReceivedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
ProcessedAt	datetime2(3)	YES	—	—
LastError	nvarchar(1000)	YES	—	—

INDEXES: UNIQUE (StripeEventId) | INDEX (Status, ReceivedAt)

CHECK: [Status] IN ('Received','Processed','Failed','Ignored')

DESIGN RULE: Verify Stripe signature BEFORE insert; UNIQUE StripeEventId prevents double handling of redeliveries.

Coupon / dbo.Coupons

COMMERCE | Promotional code and redemption eligibility.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
Code	varchar(60)	NO	—	—
DiscountType	varchar(12)	NO	—	—
DiscountValue	decimal(18,2)	NO	—	—
ValidFrom	datetime2(3)	YES	—	—
ValidUntil	datetime2(3)	YES	—	—
MaxRedemptions	int	YES	—	—
IsActive	bit	NO	—	1

INDEXES: UNIQUE (Code)

CHECK: [DiscountType] IN ('Percentage','Fixed') ; [DiscountValue] > 0 ; ([DiscountType] <> 'Percentage' OR [DiscountValue] <= 100) ; [MaxRedemptions] IS NULL OR [MaxRedemptions] > 0 ; [ValidUntil] IS NULL OR [ValidFrom] IS NULL OR [ValidUntil] > [ValidFrom]

DESIGN RULE: Application normalizes Code to uppercase; a configured DB collation should also support case-insensitive lookup.

CouponRedemption / dbo.CouponRedemptions

COMMERCE | Coupon usage audit against a single order.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
CouponId	bigint	NO	FK to Coupons.Id	—
OrderId	bigint	NO	FK to Orders.Id	—
UserId	nvarchar(450)	NO	FK to AspNetUsers.Id	—
AppliedAmount	decimal(18,2)	NO	—	—
Status	varchar(16)	NO	—	'Pending'
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: UNIQUE (OrderId) | INDEX (CouponId, Status)

CHECK: [AppliedAmount] >= 0 ; [Status] IN ('Pending','Applied','Reverted')

DESIGN RULE: A pending order does not consume a final redemption; count Applied records for campaign limits.

InstructorEarning / dbo.InstructorEarnings

COMMERCE | Signed immutable ledger entries, including refunds/reversals.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
OrderItemId	bigint	NO	FK to OrderItems.Id	—
InstructorUserId	nvarchar(450)	NO	FK to InstructorProfiles.UserId	—
EntryType	varchar(20)	NO	—	—
EventRef	varchar(100)	NO	—	—
Amount	decimal(18,2)	NO	—	—
Currency	char(3)	NO	—	—
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: UNIQUE (OrderItemId, EventRef) | INDEX (InstructorUserId, CreatedAt)

CHECK: [EntryType] IN ('Credit','Reversal','Adjustment') ; ([EntryType] = 'Credit' AND [Amount] >= 0) OR ([EntryType] = 'Reversal' AND [Amount] <= 0) OR [EntryType] = 'Adjustment'

DESIGN RULE: Use signed entries rather than overwriting revenue. Validate instructor ownership in application service.

08 Engagement domain / 5 tables

CourseReview / dbo.CourseReviews

ENGAGEMENT | One 1-to-5 star review per enrollment.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
EnrollmentId	bigint	NO	FK to Enrollments.Id	—
Rating	tinyint	NO	—	—
Body	nvarchar(2000)	YES	—	—
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
UpdatedAt	datetime2(3)	YES	—	—

INDEXES: UNIQUE (EnrollmentId)

CHECK: [Rating] BETWEEN 1 AND 5

DESIGN RULE: Average course rating is computed/aggregated, rather than independently edited on Course.

ReviewHelpfulVote / dbo.ReviewHelpfulVotes

ENGAGEMENT | Per-user helpful flag on a review.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
ReviewId	bigint	NO	PK; FK to CourseReviews.Id	—
VoterUserId	nvarchar(450)	NO	PK; FK to AspNetUsers.Id	—
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: INDEX (VoterUserId)

CourseQuestion / dbo.CourseQuestions

ENGAGEMENT | Course-specific learner Q&A post.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
CourseId	bigint	NO	FK to Courses.Id	—
AuthorUserId	nvarchar(450)	NO	FK to AspNetUsers.Id	—
Title	nvarchar(180)	NO	—	—
Body	nvarchar(max)	NO	—	—
IsResolved	bit	NO	—	0
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: INDEX (CourseId, CreatedAt)

CourseAnswer / dbo.CourseAnswers

ENGAGEMENT | Response from instructor or permitted learner.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
QuestionId	bigint	NO	FK to CourseQuestions.Id	—
AuthorUserId	nvarchar(450)	NO	FK to AspNetUsers.Id	—
Body	nvarchar(max)	NO	—	—
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()

INDEXES: INDEX (QuestionId, CreatedAt)

Notification / dbo.Notifications

ENGAGEMENT | In-app message created from application events.

COLUMN	SQL SERVER TYPE	NULL	KEY / RELATION	DEFAULT
Id	bigint	NO	PK; IDENTITY	—
RecipientUserId	nvarchar(450)	NO	FK to AspNetUsers.Id	—
Type	varchar(40)	NO	—	—
Title	nvarchar(180)	NO	—	—
Body	nvarchar(2000)	NO	—	—
CreatedAt	datetime2(3)	NO	—	SYSUTCDATETIME()
ReadAt	datetime2(3)	YES	—	—

INDEXES: INDEX (RecipientUserId, ReadAt, CreatedAt)

DESIGN RULE: ReadAt IS NULL denotes unread; no duplicate IsRead flag is stored.

05 Indexing & workload strategy

Indexes should support observed query paths without over-indexing write-heavy progress and webhook tables.

QUERY / OPERATION	LEADING INDEX KEY(S)	DESIGN GOAL
Catalog browse	Courses(CategoryId, Status)	Category + published filter
Instructor dashboard	Courses(InstructorUserId, Status)	Owner course listing
Course lesson sequence	CourseSections(CourseId, SortOrder); Lessons(SectionId, SortOrder)	Deterministic ordering
My courses / duplicate enrollment	Enrollments(StudentUserId, CourseId) UNIQUE	Entitlement lookup
Resume lesson	LessonProgresses(EnrollmentId, LessonId) UNIQUE	Idempotent progress UPSERT
Quiz retakes	QuizAttempts(QuizId, EnrollmentId, AttemptNumber) UNIQUE	No duplicate attempt number
Stripe event deduplication	PaymentWebhookEvents(StripeEventId) UNIQUE	Event processing exactly once per ID
Order history	Orders(BuyerUserId, CreatedAt)	Buyer timeline
Instructor earnings	InstructorEarnings(InstructorUserId, CreatedAt)	Ledger and reporting
Unread notifications	Notifications(RecipientUserId, ReadAt, CreatedAt)	Inbox query

Nonclustered indexes on foreign keys and the defined business keys are generated in the companion T-SQL. The Identity provider also creates its standard account and role lookup indexes. Revisit selectivity, include-columns and fragmentation after measuring representative workload, query plans and transaction rates. [R2][R5]

High-risk write contention

Enrollment creation: the UNIQUE(student,course) index makes retries safe. Progress tracking: avoid a per-video-frame SQL write; debounce resume-position updates and enforce one row per enrollment/lesson. Payments: unique provider event IDs plus transactional fulfillment are mandatory; never rely on client redirects as a paid-enrollment signal. Course editing: ROWVERSION prevents silent lost updates. [R5][R6]

06 Referential integrity & transactional rules

Database constraints handle local invariants; service-layer validation enforces cross-row and cross-system business meaning.

ERD RULE	AREA	PHYSICAL ENFORCEMENT OR SERVICE VALIDATION
R01-R03	Identity & catalog	User-role membership uses composite key. Profiles have PK = identity FK. Section and lesson SortOrder are unique per parent.
R04-R06	Enrollment	UNIQUE(StudentUserId, CourseId); OrderItemId unique if non-null. Lesson must resolve to the enrolled course.
R07-R08	Assessment	Section-linked quiz must belong to Quiz.CourseId. One answer per attempt/question; selected option must match its question.
R09-R10	Reviews / certificates	One review and one certificate per enrollment; ratings 1-5, verification codes unique.
R11-R12	Orders / payments	Unique line per order/course; purchase snapshots immutable; Stripe event and non-null provider object IDs unique.
R13-R14	Coupon / earnings	At most one coupon redemption per order; earning ledger events idempotent for order item and provider/business event.

Transaction boundaries

T1 - Paid checkout. Within one transaction, freeze Order and OrderItem price snapshots. Provider session creation follows a retry-safe orchestration; never store card numbers.

T2 - Webhook fulfillment. Verify signature, persist StripeEventId once, validate amount/currency and provider state, then mark order paid, create enrollments and post earnings idempotently within a transaction. [R6]

T3 - Learning completion. Enforce enrollment/lesson course match and save lesson progress. Emit a notification event after successful commit; completion percentage is derived.

T4 - Assessment and certificate. Validate quiz ownership, grade submissions, update completion state; generate certificate only after required course/quiz criteria and enforce UNIQUE(EnrollmentId).

CROSS-TABLE RULES NOT EXPRESSIBLE AS ORDINARY CHECK CONSTRAINTS

Enrollment.CourseId must equal its OrderItem.CourseId when paid; progress/notes/bookmarks lessons must belong to the enrollment course; quiz attempts must match the enrolled course; answers/options must belong to the submitted quiz; currency and sum of line totals must equal the order header. Implement these via domain services and atomic EF Core transactions, or redesign keys for composite FK enforcement.

07 EF Core mapping & migration plan

Implementation guidance for the three-layer .NET solution, without claiming the database is already implemented.

Identity integration

Use an IdentityDbContext<ApplicationUser, ApplicationRole, string> (or the equivalent IdentityDbContext generic overloads) so framework migrations remain authoritative for AspNetUsers, AspNetRoles, AspNetUserRoles and their auxiliary tables. EduHub domain entities can live in the same DbContext under EduHub.Data. Map domain FK references using the Identity string key type to avoid key-size mismatch. [R4]

EF Core fluent configuration examples

```
modelBuilder.Entity<Enrollment>(b =>
{
    b.HasIndex(x => new { x.StudentUserId, x.CourseId }).IsUnique();
    b.HasIndex(x => x.OrderItemId)
     .IsUnique().HasFilter("[OrderItemId] IS NOT NULL");
    b.HasOne(x => x.StudentProfile)
     .WithMany().HasForeignKey(x => x.StudentUserId)
     .OnDelete(DeleteBehavior.Restrict);
});
modelBuilder.Entity<Course>()
    .Property(x => x.RowVersion).IsRowVersion();
```

Every configured relationship should use explicit DeleteBehavior.Restrict/NoAction to match the SQL contract. Decimal columns must declare precision and scale, and status values should have a stable serialization policy. SQL Server ROWVERSION is a concurrency token, not a timestamp. [R5]

Migration delivery sequence

STEP	TEAM ACTION	REVIEW GATE
1	Configure Identity and ApplicationUser extensions.	Check key type and standard tables.
2	Implement entity classes / Fluent API and add migration.	Diff generated SQL to physical data dictionary.
3	Review constraints, filtered indexes and delete behavior.	Check R01-R14 and currency rules.
4	Run migrations on a clean test database.	All 35 domain tables and Identity tables exist.
5	Seed roles and demo data; test invalid operations.	FK, CHECK, UNIQUE and concurrency tests pass.
6	Create and review deployment SQL/migration bundle.	No unreviewed production auto-migration.

COMPANION SQL FILE

EduHub_SQLServer_Schema.sql contains CREATE TABLE, PK, FK, CHECK and CREATE INDEX statements for all 35 custom domain tables. It intentionally does not recreate the three core or four auxiliary ASP.NET Identity tables. Treat it as a reviewed reference implementation; choose EF Core Migrations as the primary schema deployment mechanism. [R7]

08 Sources, traceability & review

References and unresolved design decisions to settle before implementation sign-off.

SOURCE	COVERAGE IN THIS DOCUMENT
[S1] EduHub_ProjectDetails(1).pdf, pp. 1-5	Roles, courses, Blob media, progress, quizzes, carts, coupons, Stripe, certificates, analytics.
[S2] EduHub_ER_Diagram.pdf, pp. 2, 11-15	Exact 38-entity model, identity/catalog domains, cardinalities and R01-R14 rules.
[S3] EduHub_Software_Architecture.pdf	Three-layer NTier structure, platform and data boundaries.

[R1] Microsoft, EF Core relationships

https://learn.microsoft.com/en-us/ef/core/modeling/relationships

[R2] Microsoft, SQL Server CHECK constraints

https://learn.microsoft.com/en-us/sql/relational-databases/tables/create-check-constraints

[R3] Microsoft, filtered indexes

https://learn.microsoft.com/en-us/sql/relational-databases/indexes/create-filtered-indexes

[R4] Microsoft, ASP.NET Core Identity model

https://learn.microsoft.com/en-us/aspnet/core/security/authentication/customize-identity-model

[R5] Microsoft, EF Core concurrency tokens

https://learn.microsoft.com/en-us/ef/core/saving/concurrency

[R6] Stripe, Checkout fulfillment

https://docs.stripe.com/checkout/fulfillment

[R7] Microsoft, EF Core migrations

https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/

Team decisions still open

1. Confirm Azure SQL vs self-hosted SQL Server and actual production currency/currency policy. 2. Confirm discount stacking and coupon eligibility scope. 3. Confirm quiz passing rules, scoring, and allowed retries. 4. Confirm deletion/retention policies and whether students can enroll multiple times after revocation. 5. Decide whether media transcoding uses an external worker and whether audit/event outbox tables are introduced in a future ERD revision. 6. Confirm that instructor earnings and refunds use the ledger described here.

ACADEMIC / ENGINEERING REVIEW STATUS

The source brief is a project requirement statement, not a finalized data contract. All SQL types, sizes, table names, lifecycle status vocabularies, indexes and defaults are traceable proposals derived from the prior ERD and official platform guidance. They must be approved by the team before implementation.